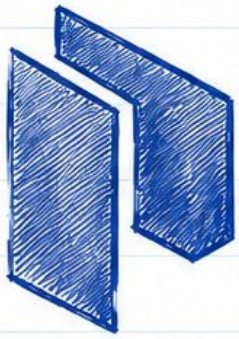
VERIQTA

VERIQTA

PACKER

FOR CLOUD ENGINEERS



BUILD AUTOMATED, SECURE,
AND REPRODUCIBLE MACHINE IMAGES
ACROSS AWS, AZURE, AND GCP

aws AWS | AZURE | GCP

- ☑ GOLDEN IMAGES
- ☑ AUTOMATED BUILDS
- ☑ SECURITY & HARDENING
- ☑ CI/CD INTEGRATION
- ☑ MULTI-CLOUD STRATEGY

FOR CLOUD ENGINEERS,
PLATFORM ENGINEERS,
AND DEVOPS PROFESSIONALS

20-PAGE PRODUCTION HANDBOOK

Veriqta YouTube · veriqta GitHub · veriqta LinkedIn · veriqta X · veriqta Instagram · veriqta Telegram

1. WHY CLOUD ENGINEERS USE PACKER

UNDERSTAND WHERE MACHINE IMAGES FIT IN MODERN CLOUD ARCHITECTURE.



WHAT PACKER SOLVES

- Automates machine image creation.
- Produces consistent, repeatable, and secure images.
- Eliminates manual server setup.
- Reduces drift and configuration snowflakes.

KEY CONCEPTS

IMAGE-BASED INFRASTRUCTURE:
Deploy servers from pre-built images.

GOLDEN IMAGES:
Hardened, tested, and approved base images.

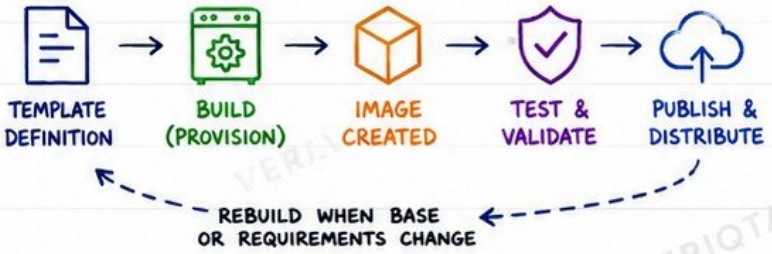
IMMUTABLE INFRASTRUCTURE:
Replace, don't modify. Images are recreated, not updated in-place.

IMAGE BAKING VS BOOT-TIME:
Bake stable dependencies into the image. Use boot-time for environment-specific settings.

PACKER VS OTHERS

TOOL	FOCUS
PACKER	Builds machine images. Creates the base.
TERRAFORM	Provisions infrastructure. Uses images to launch resources.
ANSIBLE	Configures systems. Operates on running instances.

IMAGE LIFECYCLE



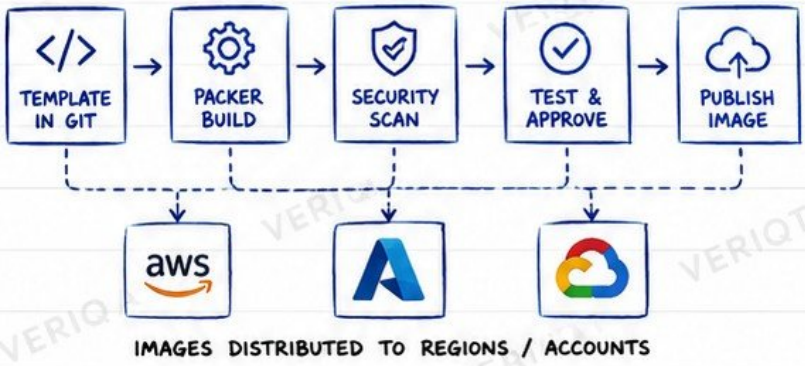
☑ WHEN IMAGE BAKING MAKES SENSE

- Stable OS and core dependencies.
- Security hardening and compliance.
- Agents, monitoring, logging, and tools.
- Applications with slow or complex installs.
- Environments where consistency is critical.

☒ WHEN IT CREATES COMPLEXITY

- Frequently changing application code.
- Environment-specific configuration.
- Secrets or dynamic runtime data.
- When teams lack an image governance process.
- Over-baking leads to large, slow, risky images.

PRODUCTION IMAGE PIPELINE OVERVIEW



SENIOR ENGINEER TIP

SEPARATE IMAGE CREATION FROM INFRASTRUCTURE DEPLOYMENT.
Let Packer own the image lifecycle.
Let Terraform (or other IaC) consume tested, approved images.

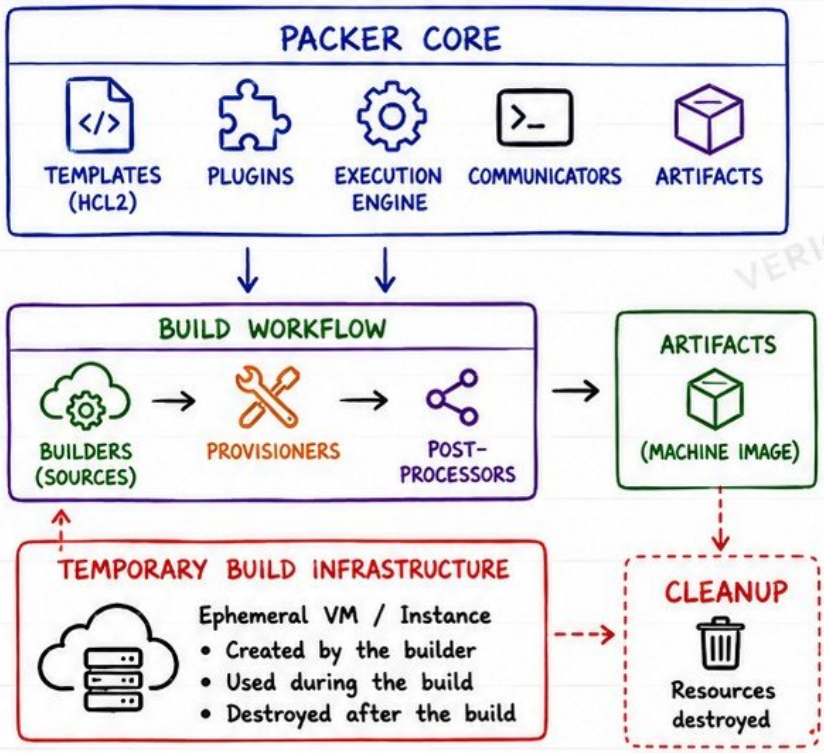
2. PACKER ARCHITECTURE AND BUILD LIFECYCLE

UNDERSTAND WHAT ACTUALLY HAPPENS WHEN PACKER BUILD RUNS.

PACKER CORE ARCHITECTURE

- **TEMPLATES (HCL2):** Define sources, builds, provisioners, and post-processors.
- **PLUGINS:** Extend Packer with builders, provisioners, and post-processors.
- **BUILDERS (SOURCES):** Create temporary infrastructure and start the build.
- **PROVISIONERS:** Configure and install software inside the instance.
- **POST-PROCESSORS:** Modify, tag, encrypt, or upload the final artifact.
- **COMMUNICATORS:** Handle connections (SSH, WinRM, etc.) between Packer and the VM.
- **ARTIFACTS:** The final machine image or output produced by the build.

HIGH LEVEL ARCHITECTURE



BUILD LIFECYCLE (FROM INIT TO ARTIFACT)

1. **INITIALIZE** — Load template, download plugins.
2. **VALIDATE** — Validate template and configuration.
3. **BUILD INFRASTRUCTURE** — Builder creates temporary VM / instance.
4. **PROVISION** — Provisioners configure the instance.
5. **POST-PROCESSS** — Post-processors modify or enhance output.
6. **CREATE ARTIFACT** — Machine image is created and stored.
7. **CLEANUP** — Temporary resources are destroyed.

CLEANUP BEHAVIOR

- ☑ Packer automatically terminates the temporary instance after a successful build.
- ☑ Attached volumes, security groups, and other resources are cleaned up.
- ☑ Temporary SSH keys and credentials are removed.
- ☑ Custom post-processors can also trigger additional cleanup.

DEBUGGING INSIGHT

WHY FAILED BUILDS CAN LEAVE CLOUD RESOURCES BEHIND

- Build interrupted before cleanup phase.
- Network or API errors prevent termination.
- Permissions missing to delete resources.
- Provisioner scripts fail and exit early.
- Custom post-processors crash.

SOLUTION: Use 'PACKER_LOG=1', enable debug, and manually clean up orphaned resources.

3. INSTALLING PACKER AND BUILDING YOUR FIRST IMAGE

BUILD A WORKING IMAGE WITHOUT TREATING THE CONFIGURATION AS A BLACK BOX.

1. GET PACKER

- Download from packer.io/downloads
- Or install via package manager.

2. VERIFY INSTALLATION

```
packer --version
```

Shows Packer version.

3. WORKFLOW COMMANDS

1. `packer init .`
   Download and install required plugins.
2. `packer fmt .`
   Format template files (HCL2 style).
3. `packer validate .`
   Validate template syntax and config.
4. `packer build .`
   Start the build and create the image.

PRODUCTION NOTE

VALIDATION SHOULD HAPPEN BEFORE CLOUD RESOURCES ARE CREATED.

ALWAYS RUN `packer validate` BEFORE `packer build`.

4. BASIC TEMPLATE FILES (.pkr.hcl)

- Packer uses HCL2 configuration files with .pkr.hcl extension.
- Templates define sources, builds, provisioners, and more.
- Keep templates modular and reusable.

REQUIRED PLUGINS

```
packer {
  required_plugins {
    amazon = {
      source = "hashicorp/amazon"
      version = "~> 1.3"
    }
  }
}
```

CORE BLOCKS

- **SOURCE BLOCKS:** Define where the image comes from and how build infrastructure is created.
- **BUILD BLOCKS:** Define what to build, provisioners, and post-processors.

5. BASIC AMAZON EBS EXAMPLE

```
source "amazon-ebs" "example" {
  region          = "us-east-1"
  source_ami      = "ami-0c02fb55956c7d316"  # Amazon Linux 2
  instance_type   = "t3.micro"
  ssh_username    = "ec2-user"
  ami_name        = "packer-demo-{{timestamp}}"
}

build {
  sources = ["source.amazon-ebs.example"]

  provisioner "shell" {
    inline = [
      "sudo yum update -y",
      "sudo amazon-linux-extras install nginx1 -y",
      "sudo systemctl enable nginx"
    ]
  }
}
```

6. UNDERSTANDING THE RESULTING AMI

- A new AMI is created from the configured instance.
- Includes OS, installed packages, and applied configurations.
- You can launch EC2 instances from this AMI.

7. KEY TAKEAWAYS

- Templates = code. Version and review them.
- Validate before build to catch errors early.
- Keep builds repeatable and deterministic.
- Understand what goes inside the image.
- Know how the image will be consumed.

4. HCL2 FOR PRODUCTION PACKER TEMPLATES

STRUCTURE TEMPLATES THAT REMAIN MAINTAINABLE AS THE IMAGE ESTATE GROWS.

1. PACKER BLOCK

```
packer {
  required_version = ">= 1.9.0"
}
```

- Sets Packer version and global settings.

2. REQUIRED_PLUGINS

```
packer {
  required_plugins {
    amazon = {
      source  = "hashicorp/amazon"
      version = "~> 1.3"
    }
  }
}
```

- Declare and pin all required plugins.

3. VARIABLES

```
variable "region" {
  type    = string
  default = "us-east-1"
}
```

- Strongly typed inputs.
- Use defaults for safe values.
- Override via var files or CLI.

COMMON VARIABLE TYPES

string	number	bool
list()	map()	object()

4. LOCALS

```
locals {
  timestamp    = formatdate("YYYYMMDDhhmm", timestamp())
  ami_prefix   = "myapp-base"
  common_tags  = {
    Project    = "myapp"
    Owner      = "platform-team"
  }
}
```

- Reusable values and computed data.
- Keep templates DRY and consistent.

5. ENVIRONMENT-DRIVEN VALUES

Use environment variables to drive behavior.

Example:

```
var.region = env("AWS_REGION", "us-east-1")
```

- Keeps templates portable across environments.

6. DATA SOURCES

```
data "amazon-ami" "al2023" {
  owners      = ["amazon"]
  most_recent = true
  filter {
    name      = "name"
    values    = ["al2023-ami-*"]
  }
}
```

- Look up dynamic information at build time.

7. EXPRESSIONS AND FUNCTIONS

- Combine values, build strings, and make logic decisions.
- Example functions:

```
format()   upper()
join()     cidrhost()
contains()
```

8. SENSITIVE VARIABLES

- Protect secrets like tokens, passwords, and keys.
- Example:

```
variable "token" {
  type      = string
  sensitive = true
}
```

9. VARIABLE FILES (.PKRVARS.HCL)

- Separate values from code.
- Example file:

```
region        = "us-west-2"
instance_type = "t3.medium"
ssh_username  = "ec2-user"
```

10. TEMPLATE ORGANIZATION

- Break templates into modules.
- Reuse blocks with locals and variables.
- Recommended structure:

```
templates/
  common.pkr.hcl          variables.pkrvars.hcl
  amazon/                 plugins.pkr.hcl
    base.pkr.hcl
  azure/
    base.pkr.hcl
```

11. NAMING CONVENTIONS

- ☑ Use clear, consistent names.
- ☑ Include OS, version, role, and date.
- ☑ Example: myapp-al2023-nginx-20240520

12. AVOID GIANT PACKER FILES

- ☑ Keep files small and focused.
- ☑ One responsibility per template.
- ☑ Use locals, modules, and includes.
- ☑ Easier to test, review, and reuse.

ENTERPRISE ADVICE

TREAT IMAGE DEFINITIONS AS PRODUCTION SOURCE CODE.

- ☑ Version control everything.
- ☑ Code review all changes.
- ☑ Test, validate, and document.
- ☑ Automate and repeat.

5. BUILDERS AND CLOUD IMAGE CREATION

UNDERSTAND HOW PACKER INTERACTS WITH CLOUD PROVIDERS.

1. BUILDER CONCEPT

- Builders know how to create temporary infrastructure and capture machine images.
- Packer talks to cloud APIs to launch, configure, and snapshot.

2. BUILDER TYPES

AMAZON EBS BUILDER
- Creates AMIs from EBS snapshots.

AZURE ARM BUILDER
- Creates managed images in Azure.

GOOGLE COMPUTE BUILDER
- Creates machine images in Google Cloud.

3. HOW PACKER WORKS WITH CLOUD PROVIDERS

SOURCE IMAGE SELECTION
- Start from a trusted base image provided by the cloud.

TEMPORARY INSTANCES
- Builder launches a temporary VM/instance for customization.

NETWORKS & SUBNETS
- Instance is placed in the specified VPC/VNet and subnets.

SECURITY GROUPS
- Firewall rules control access to the temporary instance.

SSH CONNECTIVITY
- Packer connects via SSH (or WinRM) to provision the instance.

SERVICE ACCOUNTS & IDENTITIES
- Uses IAM roles, service principals, or service accounts.

DISK CONFIGURATION
- Define volume type, size, encryption, and performance.

IMAGE CREATION
- Cloud creates machine image from the configured instance.

4. CLOUD API INTERACTIONS

- Packer uses provider APIs to:
  - ☑ Authenticate
  - ☑ Launch temporary resources
  - ☑ Configure networking and security
  - ☑ Create snapshots / images
  - ☑ Tag and register images
  - ☑ Cleanup resources

5. CLEANUP

- Packer automatically terminates temporary instances.
- Deletes attached volumes, IPs, and other resources.
- Prevents orphaned and costly leftovers.

6. PACKER BUILD FLOW (ARCHITECTURE OVERVIEW)



Reads template and config.

Launched by the builder in your cloud account.

Packer connects via SSH and installs software, configs, and updates.

Cloud creates the image (AMI / Managed Image / Machine Image).

Temporary instance and related resources are destroyed.

PRODUCTION NOTE

BUILDERS CREATE REAL CLOUD RESOURCES. ALWAYS VALIDATE, TEST, AND REVIEW CONFIGURATIONS BEFORE RUNNING BUILDS. FAIL EARLY. SAVE TIME. AVOID COSTS.

KEY TAKEAWAYS

- ☑ Builders abstract cloud complexity.
- ☑ Packer manages the full image lifecycle.
- ☑ Temporary infrastructure = short lived.
- ☑ Images are immutable artifacts.
- ☑ Cleanup keeps your environment clean and cheap.

6. BUILDING PRODUCTION AWS AMIS

MOVE FROM A DEMO AMI TO AN OPERATIONAL AWS IMAGE.

1. BASE AMI SELECTION

- Start from a trusted, maintained base AMI.
- Use the latest stable patch level.
- Prefer official or org-approved base images.

2. SOURCE_AMI_FILTER

- Use filters to always pick the correct base AMI.
- Filter by name, owner, virtualization-type, root-device-type, etc.

```
EXAMPLE
source_ami_filter = {
  filters = {
    "name" = "amzn2-ami-hvm-*-x86_64-gp2"
    "virtualization-type" = "hvm"
    "root-device-type" = "ebs"
  }
  owners = ["137112412989"] # AWS
  most_recent = true
}
```

3. OWNERS

- Use official AWS owners or your own account ID.
- Avoid "self" or public marketplace images in production.

4. INSTANCE TYPES

- Use instance types that match production hardware family.
- Ensure the type supports your workloads and region.

5. VPC & SUBNET SELECTION

- Build in a dedicated build VPC.
- Use private subnets when possible.
- Ensure proper routing and internet/NAT access.

6. IAM INSTANCE PROFILES

- Attach least-privilege IAM instance profile.
- Required for accessing AWS services during the build.

7. AMI NAMING

- Use consistent, descriptive naming.
- Include OS, version, role, and build date.

EXAMPLE: myapp-al2023-nginx-20240520-v1.2.0

8. TAGS

- Tag every AMI for ownership and purpose.
- Include environment, application, owner, cost-center, version.

COMMON TAGS

- ☑ Environment = Prod
- ☑ Application = MyApp
- ☑ Owner = Platform-Team
- ☑ CostCenter = 12345
- ☑ Version = v1.2.0
- ☑ CreatedBy = Packer

9. EBS CONFIGURATION

- Define volume type, size, and performance.
- Use gp3 or io2 for production workloads.
- Set delete_on_termination where appropriate.

```
ebs_block_device {
  device_name           = "/dev/xvda"
  volume_size           = 50
  volume_type           = "gp3"
  iops                  = 3000
  throughput            = 125
  delete_on_termination = true
}
```

10. ENCRYPTION & KMS

- Encrypt root and data volumes.
- Use your KMS CMK.
- Ensure snapshots are also encrypted.

```
kms_key_id =
"arn:aws:kms:us-east-1:11112222333:
key/abcd-1234-efgh-5678"
```

11. SNAPSHOT HANDLING

- Keep snapshots for rollback.
- Enable fast snapshot restore (FSR) if needed.
- Set snapshot retention lifecycle.

12. REGION CONSIDERATIONS

- AMIs are region-specific.
- Build in target region whenever possible.
- Replicate AMIs across regions for resilience.

13. AMI SHARING

- Share AMIs with other accounts or OU.
- Use RAM for cross-account sharing.
- Control permissions carefully.

14. CROSS-ACCOUNT STRATEGY

- Central build account creates and owns AMIs.
- Share to workload accounts.
- Use IAM + RAM + KMS to enforce boundaries.

15. PRODUCTION FAILURE

- Building from an uncontrolled upstream AMI risks:
  - ✕ Unknown changes
  - ✕ Security vulnerabilities
  - ✕ Inconsistent behavior
  - ✕ Compliance violations

ALWAYS CONTROL YOUR BASE.

PRODUCTION NOTE

CONTROL YOUR INPUTS. AUTOMATE YOUR BUILDS.
VALIDATE BEFORE DEPLOYMENT. TREAT AMIS AS CODE.

7. AZURE AND GOOGLE CLOUD IMAGE BUILDS

UNDERSTAND HOW THE SAME IMAGE ENGINEERING MODEL CHANGES ACROSS CLOUDS.

AZURE IMAGE CONCEPTS

AZURE MANAGED IMAGES:
Built from a VM. Stored as managed images in Azure.

AZURE COMPUTE GALLERY:
Store, share, and version images across regions and subscriptions.

RESOURCE GROUPS:
Images and galleries live inside resource groups.

SERVICE PRINCIPALS:
Use for authentication and automation in Packer builds.

AZURE IMAGE VERSIONS:
Each build creates a new version. Keeps history and rollback safe.

GOOGLE CLOUD IMAGE CONCEPTS

GCP MACHINE IMAGES (COMPUTE IMAGES):
Created from disks of VM instances. Reusable across the project.

PROJECTS:
Images belong to specific projects. Use projects to isolate environments.

SERVICE ACCOUNTS:
Authenticate Packer using a service account with minimal roles.

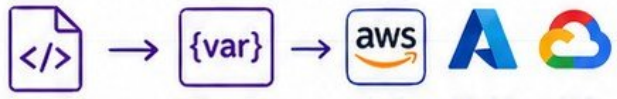
NETWORKS:
Builds run in selected VPC networks and subnets.

IMAGE FAMILIES:
Group related images. New builds create new family versions.

AWS vs AZURE vs GCP IMAGE CONCEPTS

CONCEPT	AWS	AZURE	GCP
IMAGE TYPE	AMIs	Managed Images	Compute Images
SHARING	AMI Sharing / Copy	Compute Gallery	Image Sharing / Projects
VERSIONING	New AMI ID	Image Versions	Image Family Versions
SCOPE	Region	Region / Gallery	Project / Region
AUTH	IAM Roles / Users	Service Principals	Service Accounts
STORAGE	EBS Snapshots	Managed Disks	Persistent Disks

MULTI-CLOUD TEMPLATE STRATEGY



SAME APPROACH, CLOUD-SPECIFIC IMPLEMENTATIONS

CLOUD-SPECIFIC VARIABLES

- ☑ REGION / LOCATION
- ☑ RESOURCE IDS (VPC, SUBNETS, RG, PROJECTS)
- ☑ CREDENTIAL REFERENCES
- ☑ INSTANCE / VM SIZES
- ☑ IMAGE NAMES AND TAGS
- ☑ NETWORK CONFIGURATION
- ☑ SERVICE ACCOUNTS / ROLES

ENTERPRISE ADVICE

STANDARDIZE POLICY, NOT NECESSARILY IDENTICAL IMAGES.
Use common security, hardening, and compliance baselines, but allow cloud-native differences where they add real value.

8. PROVISIONERS AND IMAGE CONFIGURATION

UNDERSTAND WHAT SHOULD HAPPEN DURING IMAGE CREATION.

TYPES OF PROVISIONERS

SHELL PROVISIONER
- Run shell scripts
- Install packages
- Configure system
- Most commonly used

FILE PROVISIONER
- Upload files
- Config templates
- Scripts
- Certificates
- Keys

ANSIBLE PROVISIONER
- Run Ansible playbooks
- Advanced config
- Idempotent
- Complex setups

POWERSHELL PROVISIONER
- Windows automation
- Install packages
- Configure Windows services

WHAT HAPPENS DURING PROVISIONING

1. **OS UPDATES**
   Apply latest patches and security updates.
2. **PACKAGE INSTALLATION**
   Install required packages and dependencies.
3. **CONFIGURATION FILES**
   Add config files, templates, and environment settings.
4. **SERVICE SETUP**
   Enable and configure services to start at boot.
5. **CLEANUP**
   Remove temp files, caches, and build artifacts.
6. **REBOOTS**
   Reboot when required. Ensure clean final state.

PROVISIONER ORDERING

Provisioners run in the order they are defined.

01 OS Updates → 02 Packages → 03 Files / Configs → 04 Services → 05 Cleanup → 06 Reboot

⚠ ORDER MATTERS. WRONG ORDER CAUSES FAILURES OR UNEXPECTED BEHAVIOR.

KEY PRINCIPLES

- ☑ IDEMPOTENCY: Provisioners should be safe to run multiple times.
- ☑ MINIMAL AND REPEATABLE: Keep steps simple and predictable.
- ☑ FAIL FAST: Stop builds on the first error.
- ☑ KEEP IMAGES CLEAN: Remove what is not needed.
- ☑ TEST CHANGES: Validate after every major change.

PACKER VS CONFIGURATION MANAGEMENT

PACKER (IMAGE BUILD TIME)	CONFIGURATION MANAGEMENT (RUNTIME)
Bakes stable dependencies	Applies environment-specific configuration
Creates immutable base images	Manages running systems
Runs once during build	Runs repeatedly after deployment
Installs OS, packages, agents	Configures apps, policies, users
Image portability	Environment customization

PROVISIONING FAILURES

- Network issues (repositories, firewall, DNS)
- Package installation errors
- Script failures / exit codes
- Authentication problems
- Reboot loops or timeouts
- Disk space or resource limits
- Debug with PACKER_LOG=1 and -on-error=ask

REAL WORLD LESSON

BAKE STABLE DEPENDENCIES,
INJECT ENVIRONMENT-SPECIFIC CONFIGURATION LATER.

- Keep images generic and reusable.
- Use Terraform/Ansible/CM for environment details.
- Smaller images, faster builds, more secure.

9. SSH, WINRM, AND COMMUNICATOR TROUBLESHOOTING

DEBUG ONE OF THE MOST COMMON SOURCES OF FAILED BUILDS.

COMMUNICATORS

SSH (LINUX / UNIX)

- Default for Linux builds
- Uses SSH over TCP 22
- Key based auth by default

WINRM (WINDOWS)

- Default for Windows builds
- Uses HTTP/HTTPS (5985/5986)
- Requires WinRM enabled

SSH LIFECYCLE

1. Packer launches a temporary VM.
2. Packer injects SSH key.
3. Packer waits for SSH to become available.
4. Provisioners run over SSH.
5. Image is created and VM is cleaned up.

KEY CONNECTIVITY FACTORS

TEMPORARY CREDENTIALS
Ephemeral keys / passwords used only during the build.

SSH USERNAMES
ubuntu, ec2-user, admin, azureuser, etc.

PRIVATE NETWORKING
Private subnets need proper routing and NAT access.

FIREWALLS & SECURITY GROUPS
Allow inbound SSH/WinRM from Packer's IP or bastion.

ROUTING
Ensure route to VM and return path to Packer host.

COMMON ISSUES

- SSH timeout / Unable to connect
- Authentication failed
- WinRM connection refused
- Wrong username or key
- Security group blocks traffic
- DNS resolution problems
- Cloud-init not finished
- Reboot during provisioning
- Network ACL or firewall denies
- Idle timeout in NAT or firewall

TROUBLESHOOTING CHECKLIST

- [x] Can Packer reach the VM network?
- [x] Is the security group / firewall correct?
- [x] Is the SSH key or WinRM auth valid?
- [x] Is the username correct?
- [x] Is cloud-init finished?
- [x] Is the port open (22 for SSH, 5985/5986 for WinRM)?
- [x] Are there routing or NAT issues?
- [x] Is the VM healthy and not rebooting?
- [x] Check Packer logs for exact failure point.

CONNECTION HANDLING

TIMEOUTS
Increase timeout when VM boot is slow.

CONNECTION RETRIES
Packer retries automatically. Tune settings if needed.

CLOUD-INIT INTERACTIONS
Provisioners may run before cloud-init completes. Add delays or use cloud-init status checks.

DEBUG MODE & LOGGING

DEBUG MODE
Run with `-debug` to see detailed step-by-step output.

```
packer build -debug template.pkr.hcl
```

PACKER_LOG
Set environment variable for detailed logs.

```
export PACKER_LOG=1
```

DEBUGGING FLOW

1. **PACKER** — Packer starts the build and initializes the plugin.
2. **NETWORK** — Check network reachability, ports, security groups, route.
3. **VM** — Verify VM is running and responding.
4. **AUTHENTICATION** — Validate SSH key or WinRM creds and username.
5. **PROVISIONING** — If connected, run provisioners and create the image.

FIX → RETRY → VERIFY EACH STEP

REAL WORLD LESSON

COMMUNICATOR PROBLEMS CAUSE MOST PACKER FAILURES.
Understand the path, validate each layer, and read the logs.
Fix the root cause, don't just increase timeouts.

10. SECRETS, CREDENTIALS, AND IAM

BUILD IMAGES WITHOUT LEAKING CREDENTIALS INTO TEMPLATES OR ARTIFACTS.

WHERE CREDENTIALS COME FROM

CLOUD CREDENTIALS
Static keys, access keys, or tokens used by Packer.

ENVIRONMENT VARIABLES
Passed to Packer or plugins at runtime.

WORKLOAD IDENTITY
OIDC / IRSA / managed identity to avoid static credentials.

IAM ROLES
Attach least-privilege roles to build instances.

SERVICE PRINCIPALS
Used in Azure for auth to build images.

SERVICE ACCOUNTS
Used in GCP to access projects and APIs.

SHORT-LIVED CREDENTIALS
Prefer tokens or keys with short expiration.

SECRET MANAGEMENT

SECRET MANAGERS
Use AWS Secrets Manager, Azure Key Vault, or GCP Secret Manager.

DYNAMIC SECRETS
Generate credentials on demand with short TTL.

ACCESS POLICIES
Restrict who/what can read each secret.

AUDIT & ROTATE
Log access and rotate secrets regularly.

SENSITIVE VARIABLES

- ☑ Use Packer variables marked as sensitive:

```
variable "db_password" {
  type      = string
  sensitive = true
}
```

- ☑ Never hardcode secrets in templates.
- ☑ Use .pkrvars.hcl or env files outside of VCS.
- ☑ Mask secrets in CI/CD logs.

BUILD-TIME SECRET HANDLING

1. Fetch secret at build time from secret manager.
2. Use secret only during provisioning.
3. Do NOT write secret to disk or persist it.
4. Remove or revoke secret after use.
5. Verify secret is not left anywhere in the image.

AVOIDING SECRETS IN IMAGE LAYERS

- ✗ Never COPY or ADD secret files into the image.
- ✗ Avoid echoing secrets in scripts or logs.
- ✗ Remember: Every layer is permanent. Secrets can be recovered.
- ✓ Clean up temp files, caches, and credentials.

LEAST PRIVILEGE PRINCIPLES

- ☑ Grant only the permissions required to build the image.
- ☑ Avoid admin or wildcard permissions.
- ☑ Use scoped roles, resource groups, or projects.
- ☑ Limit network access for build instances.
- ☑ Review permissions regularly.

CI/CD IDENTITY BEST PRACTICES

- ☑ Use dedicated CI/CD identities, not personal accounts.
- ☑ Store credentials in secure CI secret stores.
- ☑ Rotate credentials automatically.
- ☑ Validate identity permissions before builds.
- ☑ Audit who triggered each build.

SECURITY REMINDER

A SECRET REMOVED FROM A FILE MAY STILL SURVIVE ELSEWHERE IN THE IMAGE.

- Layer history
- Swap / memory
- Logs
- Shell history
- Temp files
- Backups / snapshots

11. GOLDEN IMAGE ENGINEERING

DESIGN IMAGES THAT TEAMS CAN SAFELY STANDARDIZE ON.

WHAT IS A GOLDEN IMAGE?

A pre-configured, secure, and tested machine image that serves as the standard baseline for deploying infrastructure at scale.

ORGANIZATION BASELINE

- Security hardening (CIS, STIG, etc.)
- User and access policies
- Firewall and network rules
- NTP, DNS, and system configuration
- Patch management settings
- Audit and compliance settings

KEY COMPONENTS OF A GOLDEN IMAGE

BASE OS IMAGES
Approved, stable, and supported OS.

SECURITY AGENTS
Endpoint protection, EDR, vulnerability agents.

RUNTIME DEPENDENCIES
Languages, runtimes, dependencies needed by applications.

APPROVED PACKAGES
Organization-approved software and tools only.

MONITORING AGENTS
Metrics, logs, and performance collection.

CERTIFICATES
Internal CAs, TLS certs, and trusted bundles.

LOGGING CONFIGURATION
Centralized logging, log forwarding, retention settings.

GOLDEN IMAGE LIFECYCLE

BASE IMAGE
Clean OS from vendor.

→

HARDENED IMAGE
Apply baseline, security, and organization standards.

→

APPLICATION IMAGE
Add application dependencies and runtime configs.

→

VALIDATED IMAGE
Tested, scanned, and approved for use.

OWNERSHIP, METADATA & VERSIONING

IMAGE OWNERSHIP
Every image has an owner and backup owner.

IMAGE METADATA
Tags, purpose, OS version, build date, owner, and compliance info.

IMAGE VERSIONING
Use semantic versions or timestamps. Never use "latest".

IMAGE LINEAGE
Track parent image, build pipeline, and change history.

AVOIDING IMAGE SPRAWL

- Do not create images for every team.
- Standardize base and hardened images.
- Use application layers or configuration management where possible.
- Retire outdated or unused images.
- Review and approve new image requests.

IMAGE PROGRESSION MODEL

BASE IMAGE (OS) → HARDENED IMAGE (BASELINE) → APPLICATION IMAGE (RUNTIME)

PROMOTE IMAGES THROUGH ENVIRONMENTS, NOT REBUILT FROM SCRATCH.

ENTERPRISE ADVICE

GOLDEN IMAGES NEED OWNERS, SLAS, AND RETIREMENT POLICIES.

- ☑ Define lifecycle and update cadence.
- ☑ Set SLAs for security patches and rebuilds.
- ☑ Retire images that are outdated or unused.

12. IMAGE HARDENING AND SECURITY

TURN IMAGE CREATION INTO
PART OF THE SECURITY CONTROL PLANE.

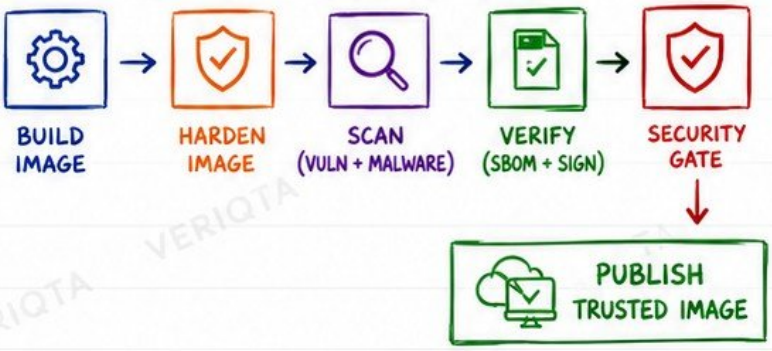
HARDENING CHECKLIST

- [x] OS HARDENING (CIS-STYLE BASELINES)
- [x] PATCH LEVELS (KEEP SYSTEM UPDATED)
- [x] PACKAGE MINIMIZATION (REMOVE UNNECESSARY)
- [x] SSH HARDENING (KEYS, PORTS, SETTINGS)
- [x] USER ACCOUNTS (LEAST PRIVILEGE ONLY)
- [x] FILE PERMISSIONS (PRINCIPLE OF LEAST ACCESS)
- [x] AUDIT CONFIGURATION (LOGGING + AUDITD)
- [x] SECURITY AGENTS (EDR, MONITORING, IDS)
- [x] DISK ENCRYPTION (AT REST PROTECTION)
- [x] VULNERABILITY SCANNING (OS + PACKAGES)
- [x] MALWARE SCANNING (DETECT MALICIOUS FILES)
- [x] SBOM CONSIDERATIONS (KNOW YOUR COMPONENTS)
- [x] IMAGE SIGNING & PROVENANCE (PROVE AUTHENTICITY)
- [x] SECURITY GATES (POLICY + COMPLIANCE CHECKS)

HARDENING PRINCIPLES

- MINIMIZE ATTACK SURFACE
- APPLY LEAST PRIVILEGE
- KEEP SYSTEMS UPDATED
- DETECT AND PREVENT THREATS
- ENFORCE COMPLIANCE STANDARDS
- PROVE TRUST AND INTEGRITY

SECURITY CONTROL PIPELINE



WHAT CAN GO WRONG

- ✗ DEFAULT USER ACCOUNTS LEFT ENABLED
- ✗ OUTDATED PACKAGES AND KERNELS
- ✗ EXPOSED PASSWORD OR KEYS
- ✗ UNNECESSARY SERVICES RUNNING
- ✗ NO SCANNING OR SIGNING
- ✗ NO POLICY ENFORCEMENT

SECURITY REMINDER

A SUCCESSFULLY BUILT IMAGE
IS **NOT** AUTOMATICALLY A TRUSTED IMAGE.

13. TESTING IMAGES BEFORE RELEASE

PREVENT BROKEN IMAGES FROM REACHING PRODUCTION.

BUILD SUCCESS vs IMAGE CORRECTNESS

BUILD SUCCESS	IMAGE CORRECTNESS
• Packer ran without errors. • Image was created. • This only proves the build worked.	• The image boots. • Software works. • Security is enforced. • Compliant and ready for production.

KEY TESTING PRINCIPLES

- A built image is NOT automatically production-ready.
- Tests must validate functionality, security, and compliance.
- Catch issues here, not after deployment.
- Automate everything you can.
- Tests should be fast, reliable, and repeatable.

WHAT TO TEST

SMOKE TESTS
Basic sanity checks after boot.

PACKAGE VERIFICATION
Ensure required packages are installed and correct versions.

SERVICE CHECKS
Verify critical services are running and healthy.

PORT CHECKS
Validate required ports are open and listening.

CONFIGURATION VALIDATION
Check files, settings, and system configuration.

SECURITY VALIDATION
Verify hardening, permissions, and security controls.

COMPLIANCE CHECKS
Ensure CIS/STIG/Org policy compliance.

IMAGE BOOT TESTS
Boot the image in a clean environment.

INFRASTRUCTURE TEST ENVIRONMENTS
Run tests in a staging environment similar to production.

AUTOMATED ACCEPTANCE TESTING
Automate tests for speed, consistency, and repeatability.

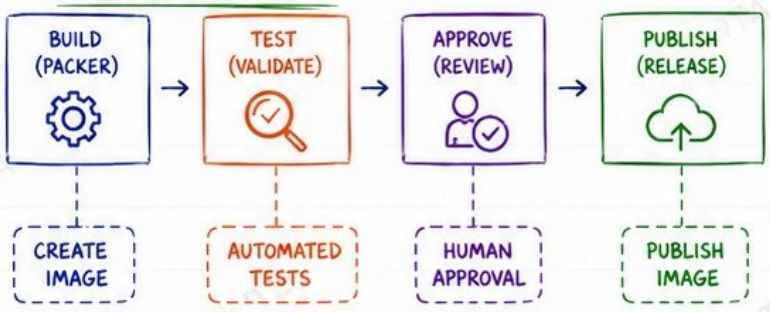
FAILURE GATES

- ✕ ANY CRITICAL TEST FAILURE = STOP
- ✕ NO IMAGE PROMOTION
- ✕ FIX, REBUILD, AND RETEST
- ✕ NO MANUAL OVERRIDE WITHOUT APPROVAL
- ✕ LOG AND NOTIFY ON FAILURE

PROMOTION CRITERIA

- ✓ ALL CRITICAL TESTS PASSED
- ✓ SECURITY AND COMPLIANCE VALIDATED
- ✓ IMAGE BOOTS AND SERVICES HEALTHY
- ✓ APPROVED BY REQUIRED REVIEWERS
- ✓ READY FOR PROMOTION TO NEXT STAGE

IMAGE RELEASE FLOW



PRODUCTION NOTE

NEVER MAKE "PACKER EXITED 0" YOUR RELEASE CRITERION.

14. PACKER IN CI/CD PIPELINES

AUTOMATE IMAGE ENGINEERING SAFELY.

KEY COMPONENTS

GIT-BASED IMAGE REPOSITORIES
Store Packer templates, variables, and pipeline configs in Git.

PULL REQUEST WORKFLOW
Every change is reviewed before images are built.

TEMPLATE VALIDATION
- packer fmt -check
- packer validate

BUILD STAGES
Build images in isolated, reproducible environments.

SECURITY SCANNING
Scan for vulnerabilities, malware, and misconfigurations.

TESTING STAGES
Run automated tests to verify functionality and compliance.

ARTIFACT PUBLISHING
Publish approved images to artifact registries.

APPROVAL GATES
Require human approval before promotion to next environment.

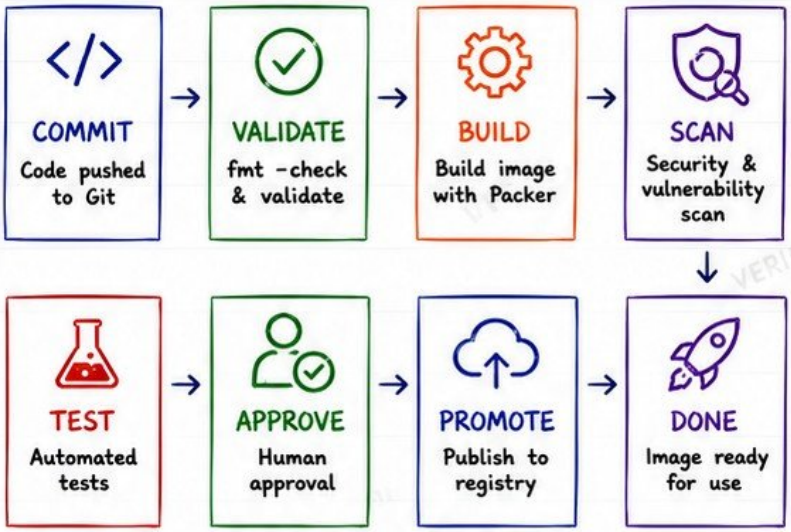
SCHEDULED REBUILDS
Automate regular image rebuilds for patching and freshness.

PIPELINE CREDENTIALS
Use short-lived credentials with least privilege.

BUILD LOGS
Store logs for auditing, troubleshooting, and compliance.

NOTIFICATIONS
Notify teams on success, failure, and approvals.

PIPELINE OVERVIEW



PIPELINE PLATFORMS

GITHUB ACTIONS JENKINS GITLAB CI

BEST PRACTICES

- ☑ KEEP TEMPLATES SMALL, MODULAR, AND REUSABLE.
- ☑ VALIDATE BEFORE YOU BUILD.
- ☑ SCAN EVERY IMAGE.
- ☑ TEST WHAT MATTERS.
- ☑ APPROVE BEFORE PROMOTION.
- ☑ REBUILD IMAGES ON A SCHEDULE.
- ☑ LOG EVERYTHING.
- ☑ NOTIFY THE RIGHT PEOPLE.
- ☑ FAIL FAST, FIX FAST.

PRODUCTION NOTE

NEVER MAKE "PACKER EXITED 0" YOUR RELEASE CRITERION.

15. IMAGE VERSIONING, PROMOTION, AND ROLLBACK

TREAT MACHINE IMAGES AS IMMUTABLE RELEASE ARTIFACTS.

VERSIONING STRATEGY

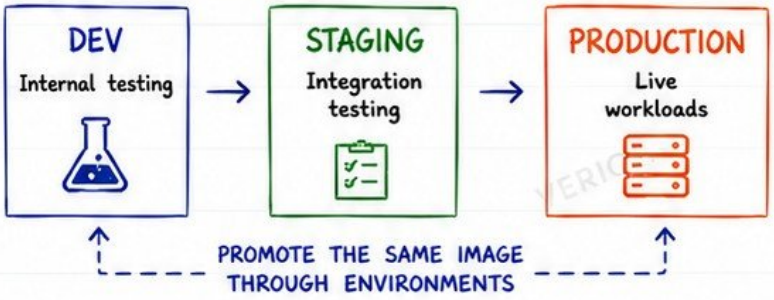
SEMANTIC VERSIONS
Major.Minor.Patch
Example: 1.4.2

TIMESTAMP VERSIONS
Date + time for uniqueness
Example: 2025.05.20.1430

BUILD IDS
Unique ID for every build
Example: build-7f3a9c1

GIT COMMIT METADATA
Store commit SHA and branch in image metadata / tags.

ENVIRONMENTAL IMAGE PROMOTION



PROMOTION vs REBUILDING

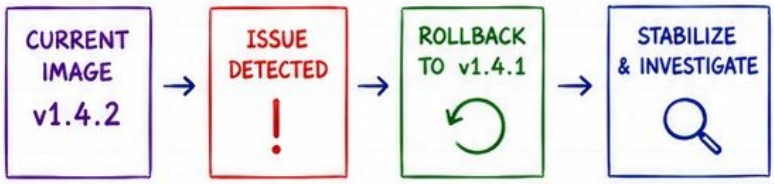
PROMOTE
- Build once, test once.
- Same image moves across environments.
- Consistent and reliable.
- Faster and traceable.

REBUILD
- Build again for each env.
- Risk of configuration drift.
- Slower and less reliable.
- Harder to trust.

IMAGE IDENTIFIERS BY CLOUD

CLOUD	IDENTIFIER
AWS	AMI ID ami-0abc123def456
AZURE	IMAGE VERSION img-2025.05.20.1
GCP	IMAGE FAMILY projects/myproj/global/images/family

ROLLBACK STRATEGY



KEEP PREVIOUS VERSIONS READY TO ROLL BACK FAST.

DEPRECATION & RETENTION

DEPRECATION
- Announce end-of-life.
- Stop creating new resources.
- Provide migration path.

RETENTION
- Keep N versions per env.
- Keep for compliance window.
- Balance cost vs safety.

BEST PRACTICES

- ☑ VERSION EVERYTHING.
- ☑ NEVER USE "LATEST" IN INFRASTRUCTURE CODE.
- ☑ PROMOTE, DON'T REBUILD.
- ☑ STORE METADATA WITH EVERY IMAGE.
- ☑ KEEP ROLLBACK PATHS ALWAYS AVAILABLE.
- ☑ AUTOMATE CLEANUP OF OLD IMAGES.
- ☑ DOCUMENT IMAGE LIFECYCLE AND OWNERSHIP.

PRODUCTION FAILURE

USING "LATEST" WITHOUT CONTROLLING WHAT LATEST MEANS WILL LEAD TO UNPREDICTABLE DEPLOYMENTS AND OUTAGES.



16. PACKER + TERRAFORM

CONNECT IMAGE PIPELINES TO INFRASTRUCTURE PROVISIONING CORRECTLY.

RESPONSIBILITY BOUNDARIES

PACKER
- BUILDS MACHINE IMAGES.
- CREATES REPRODUCIBLE ARTIFACTS.
- DOES NOT DEPLOY INFRASTRUCTURE.

TERRAFORM
- DEPLOYS INFRASTRUCTURE.
- CONSUMES EXISTING IMAGES.
- MANAGES LIFECYCLE OF RESOURCES.

RULE OF THUMB
PACKER BUILDS ARTIFACTS.
TERRAFORM DEPLOYS INFRASTRUCTURE.

IMAGE DISCOVERY OPTIONS

AMI FILTERS
Find the latest AMI by name, tags, owners, or attributes.

IMAGE IDS
Use a specific AMI / image ID for full control.

PARAMETER STORES
Store image IDs in SSM Parameter Store, Azure Key Vault, or GCP Secret Manager.

PIPELINE OUTPUTS
Pass image IDs from Packer pipeline outputs to Terraform.

TERRAFORM CONSUMPTION PATTERNS

LAUNCH TEMPLATES
Use image IDs in Launch Templates for EC2 instances.

AUTO SCALING GROUPS
ASGs reference Launch Templates which use the image.

TERRAFORM DEPENDENCIES
Use data sources and dependencies to ensure images exist before use.

WHAT TO AVOID

- ✗ DO NOT RUN PACKER BUILDS INSIDE NORMAL TERRAFORM WORKFLOWS.
- ✗ DO NOT COMPILE IMAGES AT DEPLOY TIME.
- ✗ DO NOT RELY ON "LATEST" WITHOUT CONTROL.
- ✗ DO NOT LET TERRAFORM DISCOVER IMAGES YOU DON'T CONTROL.

DEPLOYMENT FLOW

PACKER → Build and test the image.

→ **AMI / IMAGE** → Immutable artifact created.

→ **IMAGE REGISTRY** → Store and version images centrally.

→ **TERRAFORM** → Deploy infrastructure using approved image.

→ **COMPUTE** → Instances launch from the tested image.

SENIOR ENGINEER TIP

INFRASTRUCTURE DEPLOYMENT SHOULD CONSUME A TESTED IMAGE, NOT DISCOVER AN UNCONTROLLED ONE.

17. MULTI-REGION AND MULTI-ACCOUNT IMAGE DISTRIBUTION

DESIGN IMAGES FOR ENTERPRISE CLOUD ESTATES.

KEY CONCEPTS

CENTRAL IMAGE FACTORY
Build once, distribute everywhere.

SHARED-SERVICES ACCOUNTS
Security, networking, and tooling owned centrally.

AWS ORGANIZATIONS
Use OUs and SCPs for governance and guardrails.

AMI COPYING
Copy AMIs to target regions. Preserve tags and metadata.

CROSS-ACCOUNT SHARING
Share via AMI sharing or Resource Access Manager.

KMS CONSIDERATIONS
Use multi-region KMS keys or re-encrypt per region.

DISTRIBUTION OPTIONS

AWS:
- Copy AMIs to target regions.
- Share across accounts.
- Use AWS Organizations for scale.

AZURE:
- Azure Compute Gallery replication across regions.
- Image versions for consistency.

GCP:
- Share images across projects.
- Use image families for versioning.
- Replicate within/between regions.

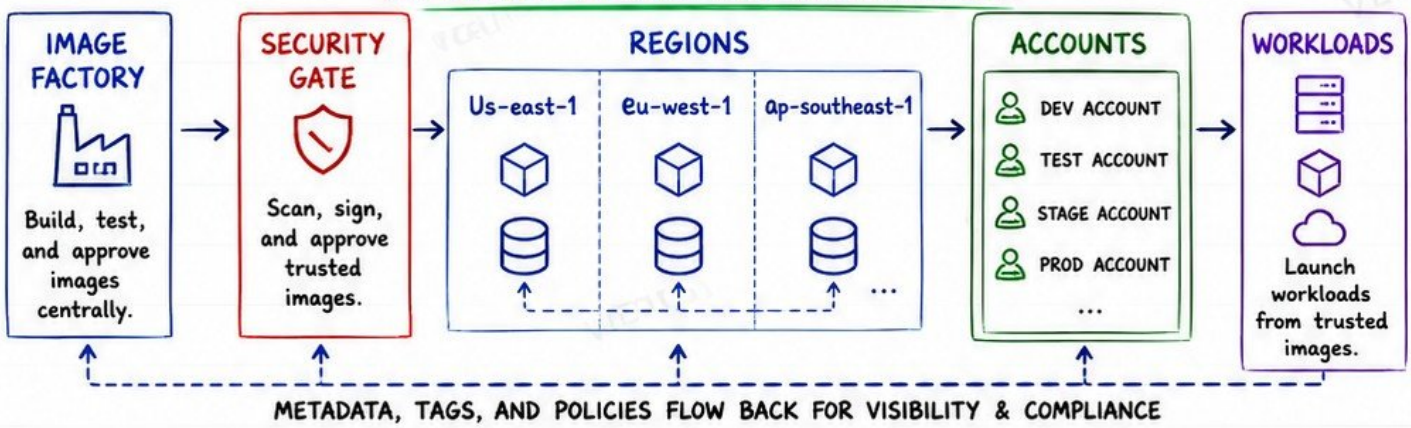
DISTRIBUTION POLICIES
Control who can copy, share, and launch images.

REGIONAL FAILURES
Maintain copies in multiple regions for resiliency.

REPLICATION STATUS
Continuously monitor replication health and drift.

GOVERNANCE BOUNDARIES
Enforce guardrails with SCPs, policies, and approvals.

ENTERPRISE DISTRIBUTION ARCHITECTURE



SCALING NOTE

DISTRIBUTION BECOMES A PLATFORM PROBLEM AT ENTERPRISE SCALE.

18. IMAGE FACTORY ARCHITECTURE

DESIGN PACKER AS AN INTERNAL CLOUD PLATFORM CAPABILITY.

WHAT IS AN IMAGE FACTORY?

AN IMAGE FACTORY IS A CENTRALIZED SERVICE THAT BUILDS, SECURES, TESTS, AND DISTRIBUTES TRUSTED MACHINE IMAGES ACROSS YOUR CLOUD ESTATE.

CORE OBJECTIVES

- [x] CONSISTENT, SECURE, AND COMPLIANT IMAGES
- [x] AUTOMATED BUILDS AND TESTS
- [x] CENTRALIZED GOVERNANCE AND POLICIES
- [x] MULTI-REGION AND MULTI-ACCOUNT DELIVERY
- [x] SELF-SERVICE WITH APPROVAL CONTROLS
- [x] FULL VISIBILITY AND AUDITABILITY

PLATFORM OWNERSHIP

- Own the image standards and policies.
- Provide self-service with guardrails.
- Operate the pipelines and scanners.
- Ensure security, compliance, and quality.
- Continuously improve and automate.

KEY COMPONENTS

SOURCE REPOSITORIES
Templates, scripts, and configuration as code.

BASE IMAGE INGESTION
Approved upstream OS images and patches.

BUILD WORKERS
Ephemeral workers that build images safely.

ISOLATED BUILD NETWORKS
Private subnets with no internet egress.

SECRET MANAGEMENT
Store and inject secrets securely.

VULNERABILITY SCANNERS
Scan images for OS and package vulnerabilities.

ARTIFACT REGISTRIES
Store and version machine images.

METADATA STORES
Image metadata, tags, SBOM, and lineage.

APPROVAL WORKFLOW
Review, test results, and manual approvals.

IMAGE CATALOGS
Discoverable images with policies and versions.

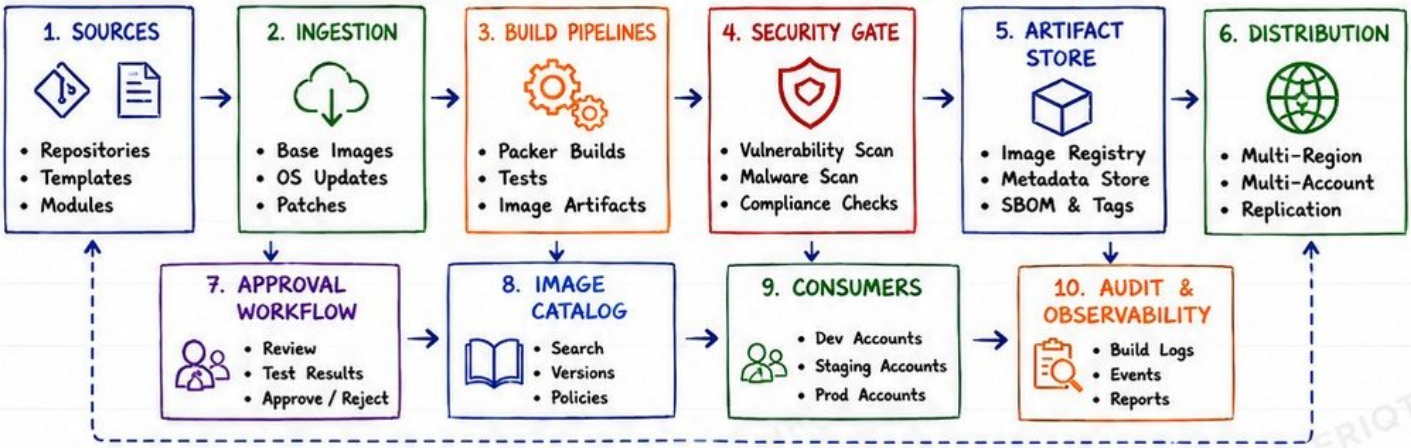
CONSUMERS
Accounts, regions, and teams using images.

AUDIT TRAILS
Every build, change, and action is recorded.

PLATFORM OWNERSHIP
Defined owners, SLAs, and operating model.

SELF-SERVICE REQUESTS
Teams request images through a portal or API.

ENTERPRISE IMAGE FACTORY ARCHITECTURE



ENTERPRISE SCALE REALITY

IMAGE FACTORY IS NOT JUST A PIPELINE. IT IS A PLATFORM SERVICE THAT REQUIRES PEOPLE, PROCESS, POLICY, AND CONTINUOUS IMPROVEMENT.

19. TROUBLESHOOTING FAILED PACKER BUILDS

DEVELOP A SYSTEMATIC PRODUCTION DEBUGGING METHOD.

COMMON FAILURE AREAS

1. TEMPLATE VALIDATION FAILURES
2. PLUGIN ERRORS
3. AUTHENTICATION FAILURES
4. API PERMISSION ERRORS
5. QUOTA FAILURES
6. SSH TIMEOUTS
7. WINRM FAILURES
8. PROVISIONER FAILURES
9. PACKAGE REPOSITORY FAILURES
10. DISK EXHAUSTION
11. REBOOT PROBLEMS
12. SNAPSHOT FAILURES
13. IMAGE CREATION FAILURES
14. CLEANUP FAILURES

DEBUG TOOLS & TECHNIQUES

PACKER_LOG

Enable detailed logs for deep visibility.

```
export PACKER_LOG=1
```

DEBUG MODE

Run in debug mode for verbose output.

```
packer build -debug template.pkr.hcl
```

TEMPORARY RESOURCE INSPECTION

Inspect the temporary instance before cleanup.

```
# Prevent cleanup
packer build -on-error=ask
# Or pause in provisioner (shell provisioner)
```

LOG COLLECTION

Collect instance logs, cloud logs, and system logs.

- /var/log/packer.log
- cloud-init logs
- boot logs
- provisioner logs

CLOUD CONSOLE

Check cloud events, syslogs, and instance status.

- CloudTrail / Activity Logs
- Instance console output
- System logs

QUICK CHECKS

- ☑ CREDENTIALS VALID?
- ☑ PERMISSIONS SUFFICIENT?
- ☑ REGION & ENDPOINTS?
- ☑ QUOTAS AVAILABLE?
- ☑ NETWORK CONNECTIVITY?
- ☑ SECURITY GROUPS?
- ☑ REPOSITORIES ACCESSIBLE?
- ☑ DISK SPACE AVAILABLE?

DEBUGGING METHODOLOGY (FOLLOW THIS ORDER)

1. **CONFIG** — Validate syntax, variables, and logic.
2. **CREDENTIALS** — Verify identities and access permissions.
3. **CLOUD API** — Check API connectivity and error responses.
4. **NETWORK** — Validate routing, security groups, and endpoints.
5. **VM** — Instance launched and running correctly?
6. **COMMUNICATOR** — SSH / WinRM connection and stability.
7. **PROVISIONER** — Scripts, packages, configs, and commands.
8. **ARTIFACT** — Image creation, snapshots, and cleanup.

PRODUCTION NOTE

ALWAYS FIND THE FIRST REAL FAILURE.
FIX THE ROOT CAUSE, NOT THE SYMPTOM.

20. PRODUCTION PACKER ARCHITECTURE AND OPERATIONAL CHECKLIST

BRING THE COMPLETE CLOUD ENGINEER WORKFLOW TOGETHER.

PRODUCTION PACKER ARCHITECTURE

1. **GIT REPOSITORY**
   Source templates, modules, and configurations.
2. **CI/CD PIPELINE**
   Automate builds with quality gates and approvals.
3. **PACKER VALIDATION**
   Syntax check, format, validate, and security lint.
4. **ISOLATED IMAGE BUILDER**
   Ephemeral, isolated environment.
   - OS HARDENING
   - PACKAGES
   - MONITORING
   - SECURITY AGENTS
5. **SECURITY + VULNERABILITY SCAN**
   Scan for vulnerabilities, malware, and compliance.
6. **AUTOMATED IMAGE TESTS**
   Functional, security, and configuration tests.
7. **APPROVED IMAGE REGISTRY**
   Store only approved, signed, and versioned images.
   - AWS ACCOUNTS / REGIONS
   - AZURE GALLERIES
   - GCP PROJECTS
8. **TERRAFORM**
   Consume approved images as data sources.
9. **PRODUCTION INFRASTRUCTURE**
   Deploy workloads using trusted, governed machine images.

OPERATIONAL CHECKLIST

- [x] IMAGE OWNERSHIP
- [x] VERSIONING
- [x] PATCH CADENCE
- [x] SECURITY GATES
- [x] TESTING REQUIREMENTS
- [x] IAM BOUNDARIES
- [x] DISTRIBUTION CONTROLS
- [x] OBSERVABILITY
- [x] DEPRECATION POLICY
- [x] ROLLBACK READINESS
- [x] AUDITABILITY
- [x] COST CLEANUP
- [x] IMAGE FRESHNESS
- [x] DISASTER RECOVERY

PRODUCTION READINESS CHECKLIST

FINAL REAL WORLD LESSON

TREAT MACHINE IMAGES AS GOVERNED SOFTWARE ARTIFACTS, NOT VM TEMPLATES.

CONSISTENT AND SECURE IMAGES | FASTER DEPLOYMENTS | GOVERNED AND COMPLIANT | REDUCED RISK | LOWER COSTS AND WASTE | READY FOR SCALE